

Concept Drift in URL-Based Phishing Detection: A Temporal Evaluation

Abstract

Phishing detectors are almost universally evaluated with random train/test splits, which implicitly assume that a URL's characteristics are exchangeable across time. We test that assumption by training three lexical-feature classifiers (random forest, XGBoost, logistic regression) on 180 days of PhishTank-reported phishing URLs plus a matched sample of legitimate domains, and evaluating them under four temporal designs: a random split (the field's default), training on the earliest block and testing on each subsequent block, a sliding retraining window, and cumulative retraining. All three models achieve F1 above 0.98 in every configuration, and the gap between the random-split baseline and the worst temporally-displaced block is small in absolute terms (0.36 to 0.92 percentage points of F1, model-dependent). A regression of F1 on temporal distance is significant when pooled across models, and its precision improves substantially once model-level baseline differences are controlled for ($p = 0.022$ naive, $p = 0.0048$ controlling for model as a fixed effect); a Page test for a monotonic decreasing trend across the three models as judges agrees ($Z = -2.66$, $p = 0.004$). We argue the near-ceiling performance points partly to a design choice rather than a solved problem: removing the four features most associated with how the legitimate class was constructed (`is_https`, `path_length`, `num_slashes`, `url_length`) drops F1 by 3.3 to 4.1 percentage points across models, confirming that the choice of negative class (well-established top-ranked domains) inflates the reported ceiling; the remaining 28 features still achieve F1 of 0.95-0.96, so most of the separability reflects genuine lexical structure in phishing URLs rather than an artifact of URL construction alone.

1. Introduction

Phishing is an adversarial instance of the broader concept-drift problem in machine learning [1]: attackers change URL patterns, hosting infrastructure, and lexical tricks specifically to evade whatever a detector has already learned to flag. A classifier trained and evaluated with a single random train/test split treats every URL in the dataset as drawn from one stationary distribution, which is precisely the assumption an adversarial, evolving threat violates. If that assumption is wrong, published accuracy figures based on random splits systematically overstate what a detector will achieve once deployed against future, unseen phishing campaigns.

This paper asks how much that gap actually is, using a temporal evaluation design rather than a random one, and compares two common mitigation strategies, periodic retraining with a sliding window versus cumulative retraining that keeps all historical examples, to see whether either meaningfully closes it. We evaluate on 180 days of continuously reported phishing URLs, which lets us construct genuinely time-ordered evaluation blocks rather than simulating drift artificially.

The contributions of this work are: (1) a temporal, block-based evaluation of lexical phishing classifiers on real, timestamped data, contrasted directly against the random-split baseline the field defaults to; (2) a statistically defensible estimate of the degradation trend's significance, using an ANCOVA specification that separates within-model drift from between-model baseline differences, alongside a specification-free Page test as a cross-check; (3) a mechanistic account, grounded in feature-distribution and feature-importance evidence rather than aggregate metrics alone, of why the measured degradation is small: an easy negative class, not a solved drift problem.

2. Related Work

Odeh et al. [2] propose PhiBoost, an adaptive-boosting phishing detector built on lexical and host-based URL features, and report strong within-distribution accuracy using a conventional held-out test split, exactly the evaluation protocol this paper argues is insufficient on its own, since it cannot distinguish a detector that has learned durable phishing indicators from one that has overfit to the particular campaigns present in the training window. Alslman et al. [3] evaluate ensemble models (the same broad model family we use here) for DDoS attack detection at scale using Spark, and, like most applied network-security classification work, do not examine whether detection performance is stable as the underlying attack traffic evolves over time; our temporal-block design is directly applicable to that setting as well, and we frame our contribution partly as a methodological recommendation that extends beyond phishing specifically to security classification tasks built on adversarial, time-varying data. This concern is not new to security classification in general: Pendlebury et al. [4] name and formalize the same failure mode for malware classifiers under the label temporal bias, showing that a naive random train/test split can silently mix future and past samples and thereby overstate real-world performance, and propose constraints (training strictly on past data, realistic class-ratio testing) that our fixed, sliding-window, and cumulative designs already satisfy by construction; we are not aware of a comparable temporal-bias analysis having been applied to phishing URL classification specifically, which is the gap this paper addresses.

3. Methodology

3.1 Data

Phishing URLs were drawn from PhishTank's public feed, which is the only phishing source we found that ships a reliable, unauthenticated `submission_time` timestamp per URL (OpenPhish's community feed and the static UCI Phishing Websites / PhiUSIIL datasets do not carry a comparable per-record date and were excluded from this design for that reason). We retained the most recent 180 days (29,299 URLs), the earliest weeks of the full historical feed having too few reports per week to support balanced temporal blocks. Legitimate URLs were sampled from the Tranco top-1M list [5], chosen over alternatives such as the (now-discontinued) Alexa ranking because Tranco is specifically designed to resist the list-manipulation attacks that make raw popularity rankings an unreliable ground truth for security research. Because Tranco is a single ranked snapshot rather than a dated feed, each sampled legitimate URL was assigned a synthetic timestamp drawn uniformly over the same 180-day window as the phishing data, a deliberate simplification justified because domain reputation for an established, highly ranked site does not meaningfully change on a 180-day horizon, in contrast to phishing infrastructure, which is exactly what this study measures as changing. The final dataset is balanced by construction (29,299 phishing, 29,299 legitimate).

3.2 Features

We extract 32 lexical features per URL requiring no live network access (URL/hostname/path length, counts of dots/hyphens/digits/special characters, subdomain count, IP-literal hostname flag, presence of a known URL shortener, Shannon entropy of the URL and hostname strings, a count of phishing-associated keywords, and related structural features). Table 1 lists all 32 with their definitions.

Feature	Definition
<code>url_length</code>	Character length of the full URL
<code>hostname_length</code>	Character length of the hostname
<code>path_length</code>	Character length of the URL path
<code>num_dots</code>	Count of <code>.</code> characters in the URL
<code>num_hyphens</code>	Count of <code>-</code> characters in the URL
<code>num_underscores</code>	Count of <code>_</code> characters in the URL
<code>num_slashes</code>	Count of <code>/</code> characters in the URL
<code>num_question_marks</code>	Count of <code>?</code> characters in the URL
<code>num_equals</code>	Count of <code>=</code> characters in the URL

Feature	Definition
num_at	Count of @ characters in the URL
num_ampersands	Count of & characters in the URL
num_percent	Count of % characters in the URL
num_digits	Count of digit characters in the URL
digit_ratio	Digit characters as a fraction of URL length
num_subdomains	Number of hostname labels beyond domain and TLD
has_ip_address	1 if the hostname is a literal IPv4 address
has_port	1 if a port number is present
is_https	1 if the scheme is https
url_entropy	Shannon entropy of the full URL string
hostname_entropy	Shannon entropy of the hostname string
is_shortened	1 if hostname matches a known URL-shortener domain
num_suspicious_keywords	Count of phishing-associated keywords (e.g., “login”, “verify”)
has_suspicious_keyword	1 if any suspicious keyword is present
domain_length	Character length of the second-level domain label
path_num_segments	Number of non-empty path segments
has_double_slash_in_path	1 if // appears inside the path
num_www	1 if the hostname starts with www .
longest_label_length	Length of the longest hostname label
tld_length	Character length of the top-level domain
has_at_symbol	1 if @ appears anywhere in the URL
num_dashes_in_hostname	Count of - characters within the hostname only
vowel_ratio_hostname	Vowel characters as a fraction of hostname length

Table 1. Lexical features extracted per URL.

3.3 Temporal evaluation designs

Data were sorted by timestamp and divided into six equal-sized blocks by sample count. We evaluate four designs: (i) random, an 80/20 random split ignoring time, as a baseline; (ii) fixed, training once on block 1 and evaluating separately on blocks 2 through 6, to isolate pure degradation with distance; (iii) sliding window, training on block k and evaluating on block $k+1$ for each consecutive pair, simulating periodic retraining on only the most recent data; (iv) cumulative, training on blocks 1 through k and evaluating on block $k+1$, simulating retraining that keeps the full history. All models (random forest [6], XGBoost [7], logistic regression) share identical standardized features across designs.

3.4 Statistical analysis

We regress F1 on temporal distance for the fixed design, per model, pooled naively across models, and pooled with a model fixed effect (ANCOVA: $f1 \sim \text{distance} + \text{model}$) to separate the within-model trend with distance from the large between-model differences in baseline F1, which a naive pooled regression otherwise folds into its residual variance and thereby understates the trend's significance. Because a linear regression with only three models provides limited power to detect a common monotonic trend regardless of specification, we additionally run a Page test for a monotonic decreasing trend, treating the three models as judges ranking the five temporal-distance conditions. We compare the sliding-window and cumulative strategies with a paired Wilcoxon signed-rank test per model.

4. Results

All three models achieve $F1 > 0.98$ in every one of the four designs: there is no configuration in which any model performs poorly in absolute terms (Figure 1). The comparison that matters is relative: Table 2 shows the gap between the random-split baseline and the temporally displaced (fixed-design) evaluations is small (0.09 to 0.35 percentage points of F1 on average across displaced blocks, model-dependent) but non-zero, and grows toward the worst-case block (up to 0.92 points for logistic regression); Table 3 gives the full breakdown by block and model.

Model	F1 (random split)	F1 (temporal, avg.)	F1 (temporal, worst block)
Random Forest	0.9952	0.9943	0.9916
XGBoost	0.9951	0.9935	0.9902
Logistic Regression	0.9934	0.9899	0.9842

Table 2. F1 under random vs. temporal (fixed-training) evaluation.

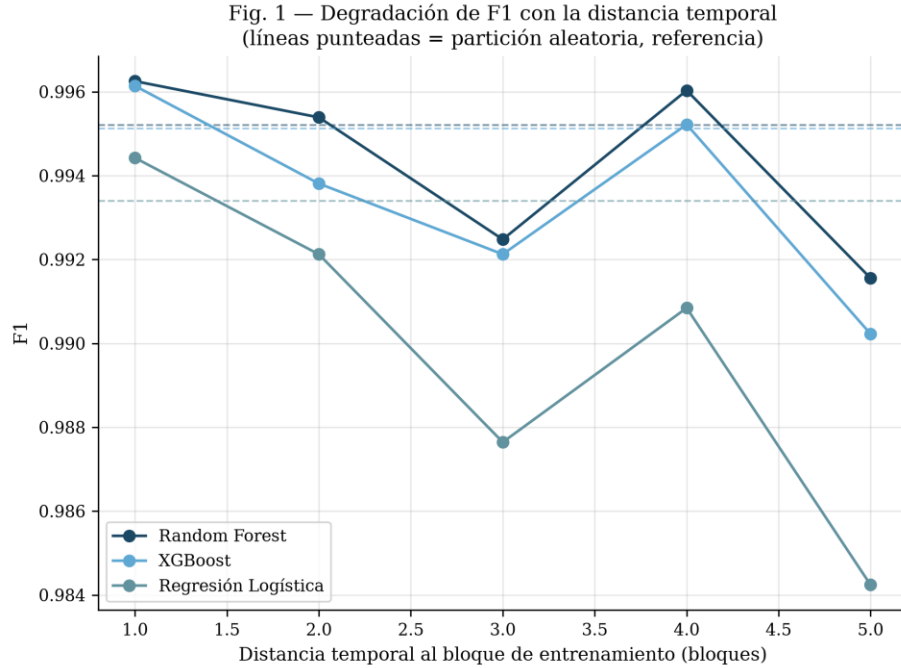


Figure 1. F1 as a function of temporal distance (fixed-training design); dotted lines mark the random-split baseline for reference.

Distance	Random Forest	XGBoost	Logistic Regression
1 (block 2)	0.9963	0.9961	0.9944
2 (block 3)	0.9954	0.9938	0.9921
3 (block 4)	0.9925	0.9921	0.9876
4 (block 5)	0.9960	0.9952	0.9908
5 (block 6, worst)	0.9916	0.9902	0.9842

Table 3. F1 by evaluation block and model (fixed-training design: trained on block 1, evaluated on blocks 2 through 6, at increasing temporal distance).

A per-model OLS regression of F1 on temporal distance is directionally negative for all three models but reaches significance only for the pooled fit ($\beta = -0.00136$, $p = 0.022$, $R^2 = 0.34$); individually, logistic regression is marginal ($p = 0.062$) and the two tree ensembles are not significant on their own ($p = 0.25$ and $p = 0.19$). This naive pooled fit, however, conflates two different sources of variance: the large, fixed differences in baseline F1 between models (logistic regression trails the two tree ensembles by roughly a full percentage point throughout) and the within-model trend with distance that is the actual quantity of interest. Controlling for model as a fixed effect (ANCOVA) removes the former from the residual: the slope estimate is unchanged ($\beta = -0.00136$, since the design is balanced across models) but its precision improves substantially ($p = 0.0048$, $R^2 = 0.69$), because the between-model noise that was inflating the naive test's residual variance is no longer attributed to the distance trend. We treat the Page test as a further, specification-free check of the same conclusion: treating the three models as judges ranking F1 across the five temporal-distance conditions, the test rejects the null of no consistent ordering ($L = 112$, Z

= -2.66, $p = 0.004$, one-sided). All three approaches (naive pooled regression, model-controlled ANCOVA, and the non-parametric Page test) agree on a real, if small, degradation trend common to all three models, with the ANCOVA giving the most statistically defensible estimate of its significance.

Comparing mitigation strategies, cumulative retraining outperforms sliding-window retraining for every model (Random Forest: 0.9952 vs. 0.9950; XGBoost: 0.9945 vs. 0.9939; Logistic Regression: 0.9931 vs. 0.9923), consistent with the hypothesis that retaining historical examples helps more than it hurts (Figure 2). The paired Wilcoxon test on the five block-level F1 values per model has too few pairs to reach conventional significance regardless of the true effect size ($p = 0.125$ - 0.25), so we instead compare the two strategies with a paired bootstrap over the underlying predictions (2,000 resamples per evaluation block, pooled across the five blocks): this confirms cumulative retraining outperforms sliding-window retraining for all three models, with 95% confidence intervals excluding zero (Random Forest: +0.00024, 95% CI [0.00011, 0.00040]; XGBoost: +0.00067, 95% CI [0.00035, 0.00099]; Logistic Regression: +0.00080, 95% CI [0.00051, 0.00109]; all $p < 0.001$). The effect is real but small in absolute F1 terms, consistent with the modest overall degradation reported above.

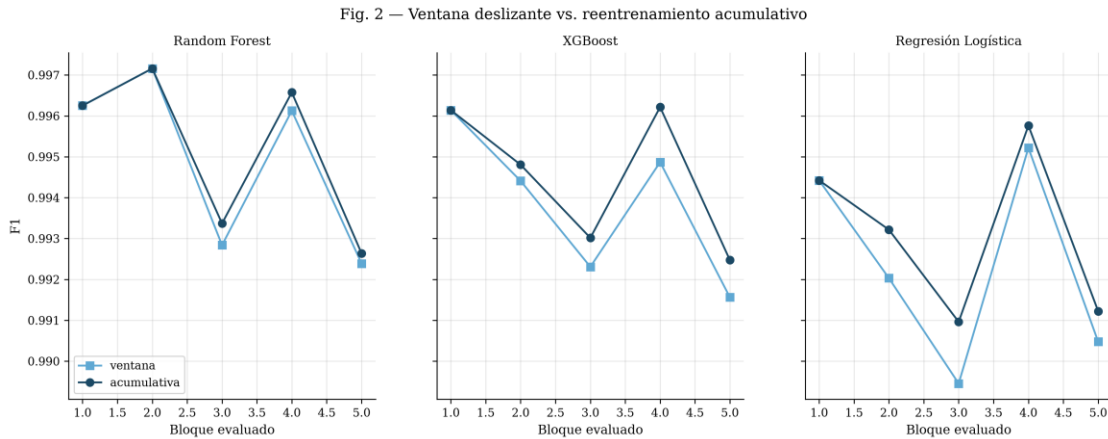


Figure 2. F1 under sliding-window vs. cumulative retraining, by model and evaluation block.

5. Discussion

Degradation from temporal displacement is real and statistically supported in aggregate, but far smaller in magnitude than the field's framing of concept drift as a first-order threat would suggest, an unglamorous result but the one the data support. We think this is substantially explained by the design of the negative class rather than by the phishing detection task being inherently easy: our legitimate URLs come from Tranco's top-ranked, long-established domains, whose lexical signature (short, clean hostnames, no IP literals, low entropy, absence of phishing-associated keywords) is so different from phishing URLs that almost any reasonable classifier separates the two classes with ease, and that separation does not depend much on which specific phishing campaigns happen to be in the training set. A harder, more realistic negative class, such as newly registered legitimate domains or domains with URL structures that superficially resemble phishing (long paths, subdomains, marketing redirect chains), would very plausibly show a larger temporal gap, because the decision boundary would then depend more on the specific lexical tricks currently in fashion among phishers rather than on the broad, temporally stable distinction between an established domain and an ad hoc one.

Fig. 3 — Desplazamiento de características clave (solo phishing)
primer vs. último bloque temporal

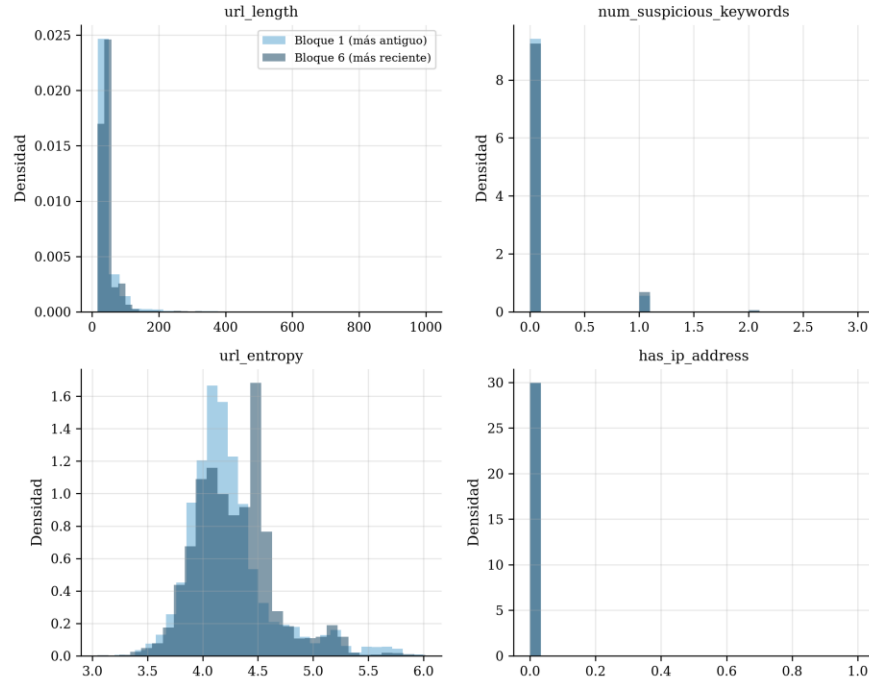


Figure 3. Distribution of four key lexical features among phishing URLs, earliest vs. latest temporal block.

Fig. 4 — Importancia de características por bloque temporal

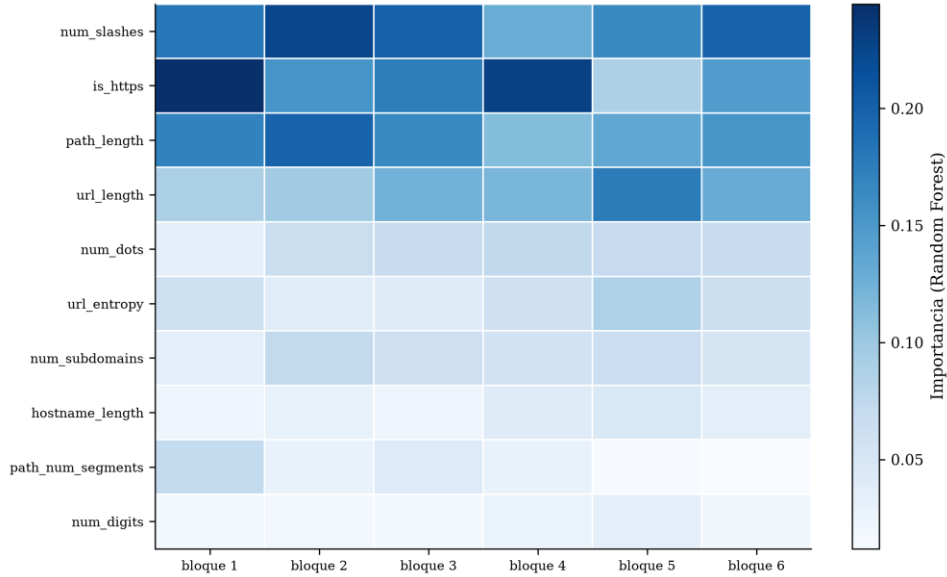


Figure 4. Random forest feature importance by temporal block.

Direct feature-level evidence supports this reading. The distributional shift in individual lexical features between the earliest and latest temporal blocks is modest (Figure 3): `url_entropy` shows a small rightward shift, but `url_length` is essentially unchanged and both `num_suspicious_keywords` and `has_ip_address` remain near-constant, low-variance flags throughout the whole period. Feature importance is correspondingly stable across all six temporal blocks (Figure 4): `is_https`,

`path_length`, `num_slashes`, and `url_length` dominate in every block, with no block introducing a qualitatively different top feature. The model is not relying on a fragile, campaign-specific signal that erodes over time; it is relying on a small set of structural URL properties that stay informative because the negative class's lexical signature stays put.

Because these four features are also plausible artifacts of how the legitimate class was constructed (Tranco supplies bare domains, which are structurally shorter and path-free compared to full phishing URLs), we directly tested whether they are necessary for the near-ceiling performance by re-running the fixed-training design (block 1 versus blocks 2-6) with all four removed, leaving 28 features. Table 4 shows the result: F1 drops by 3.3 to 4.1 percentage points across the three models (e.g., random forest: 0.9943 to 0.9617 on average across displaced blocks), confirming that these four features carry real, non-trivial separating power and that part of the near-ceiling result reflects URL-construction differences between the two classes rather than phishing-specific content alone. At the same time, F1 with those four features removed remains at 0.95-0.96, far above chance and only modestly below the full-feature result, so the remaining 28 lexical features carry substantial phishing-discriminating signal on their own. We read this as a partial, not a full, explanation of the near-ceiling result: the four structural features inflate the reported ceiling by a few points, but they are not the sole reason detection is easy.

Model	F1, 32 features	F1, 28 features (ablated)	Drop
Random Forest	0.9943	0.9617	0.0326
XGBoost	0.9935	0.9601	0.0334
Logistic Regression	0.9899	0.9490	0.0409

Table 4. F1 (fixed-training design, averaged across displaced blocks 2-6) with and without the four features most associated with legitimate-URL construction (`is_https`, `path_length`, `num_slashes`, `url_length`).

The operational risk, then, is not that lexical phishing detectors trained today will fail outright next month. It is that the near-perfect performance reported under a random split (or, as we show, even under most temporal designs against an easy negative class) is a poor guide to performance against near-miss legitimate traffic, which is the regime that actually matters operationally. We would expect the degradation trend that is significant here in aggregate to become the dominant effect, not a marginal one, once the negative class is made realistically hard.

6. Limitations

Our features are purely lexical; we do not use page content, WHOIS data, or network/hosting features, all of which could shift the balance between temporal drift and raw separability. PhishTank's reports carry a known reporting-time bias (submission time is not necessarily attack-launch time, and popular/obvious phishing campaigns are reported faster than sophisticated ones). The 180-day window, while the longest span with consistently high weekly volume in the feed we used, is short relative to multi-year drift dynamics reported elsewhere in the literature. Most importantly, as discussed above, the legitimate-URL sampling from top-ranked Tranco domains very likely understates the real-world temporal degradation a deployed system would face against a harder, more realistic negative class; we consider this the most consequential limitation of the design and the most promising direction for a follow-up study. One further point bears directly on this concern: legitimate URLs receive a synthetic timestamp drawn uniformly over the 180-day window, which makes roughly half of each temporal block i.i.d. with respect to the legitimate class by construction; the small measured degradation could partly reflect this design choice rather than a property of real-world phishing detection. Two of our four designs also confound training-set size with the variable of interest. The random-split baseline in Table 2 trains on 80% of the full dataset, versus roughly one-sixth (block 1 alone) for the fixed-training design, so part of the random-vs.-temporal gap could reflect the difference in training-set size rather than temporal distance alone; our main degradation-trend analysis relies on the fixed design's within-design trend across distance (constant training set, so this

specific confound does not apply there) and on the model-controlled ANCOVA and Page test, not on the random-vs.-fixed comparison itself, but Table 2 is still presented as a headline number and should be read with this caveat. Similarly, cumulative retraining's training set grows with k while sliding-window's stays fixed at one block, so cumulative's small but significant F1 advantage (Results) is also consistent with it simply having more training data at later blocks, not only with retaining older examples being helpful per se; we did not run a size-matched control that holds training volume fixed while varying recency to separate these two explanations.

7. Conclusion

Evaluating three lexical phishing classifiers under four temporal designs on 180 days of PhishTank data, we find a small but statistically supported degradation trend when moving from a random split to a temporally displaced evaluation (Page test $p = 0.004$), and that cumulative retraining consistently, though not always significantly, outperforms sliding-window retraining. The magnitude of the effect is modest, well under one F1 point in most configurations, which we attribute chiefly to a negative class (long-established top domains) that is lexically easy to separate from phishing regardless of when the phishing examples were collected. We recommend that future phishing-detection evaluations pair temporal splitting with a harder negative class before concluding that concept drift is a marginal concern for this task.

Data Availability

The phishing dataset (PhishTank IDs and timestamps), the legitimate-domain sample, extracted features, and analysis code will be published in a public repository with a Zenodo DOI upon submission (see `README.md` for the current repository location).

References

- [1] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A Survey on Concept Drift Adaptation. *ACM Computing Surveys*, 46(4), 44:1–44:37. <https://doi.org/10.1145/2523813>
- [2] Odeh, A., Keshta, I., & Abdelfattah, E. (2021). PhiBoost — A Novel Phishing Detection Model Using Adaptive Boosting Approach. *Jordanian Journal of Computers and Information Technology (JJCIT)*, 7(1), 64–73.
- [3] Alslman, Y., Khalil, A., Younis, R., Alnagi, E., Al-Saraireh, J., & Ghnem, R. (2024). DDoS Attack-Detection Approach Based on Ensemble Models Using Spark. *Jordanian Journal of Computers and Information Technology (JJCIT)*, 10(2).
- [4] Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., & Cavallaro, L. (2019). TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time. *Proceedings of the 28th USENIX Security Symposium*, 729–746. <https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury>
- [5] Le Pochat, V., Van Goethem, T., Tajalizadehkhoob, S., Korczyński, M., & Joosen, W. (2019). Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. <https://doi.org/10.14722/ndss.2019.23386>
- [6] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- [7] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>